

Примеры тестов на C++



Перед работой с примерами тестов, ознакомьтесь с [инструкцией по юнит-тестам!](#)

При разработке тестов на C++ следует учитывать тот факт, что протестированы могут быть только экспортируемые функции.

1. Запуск [CMake](#) - для генерации проектных файлов:

- Windows:

```
1 cmake ..\my_project\tests\<имя тестового проекта>\src ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-
  i686.cmake"
```

- Linux и macOS на Intel (x86_64):

```
1 cmake ../my_project/tests/<имя тестового проекта>/src \
2 -DSBIS_UNIT_TESTS=1 -G Ninja
```

- macOS на Apple M1 (ARM):

```
1 cmake ../my_project/tests/<имя тестового проекта>/src \
2 -DSBIS_ARCHITECTURE=arm64 \
3 -DSBIS_UNIT_TESTS=1 -G Ninja
```

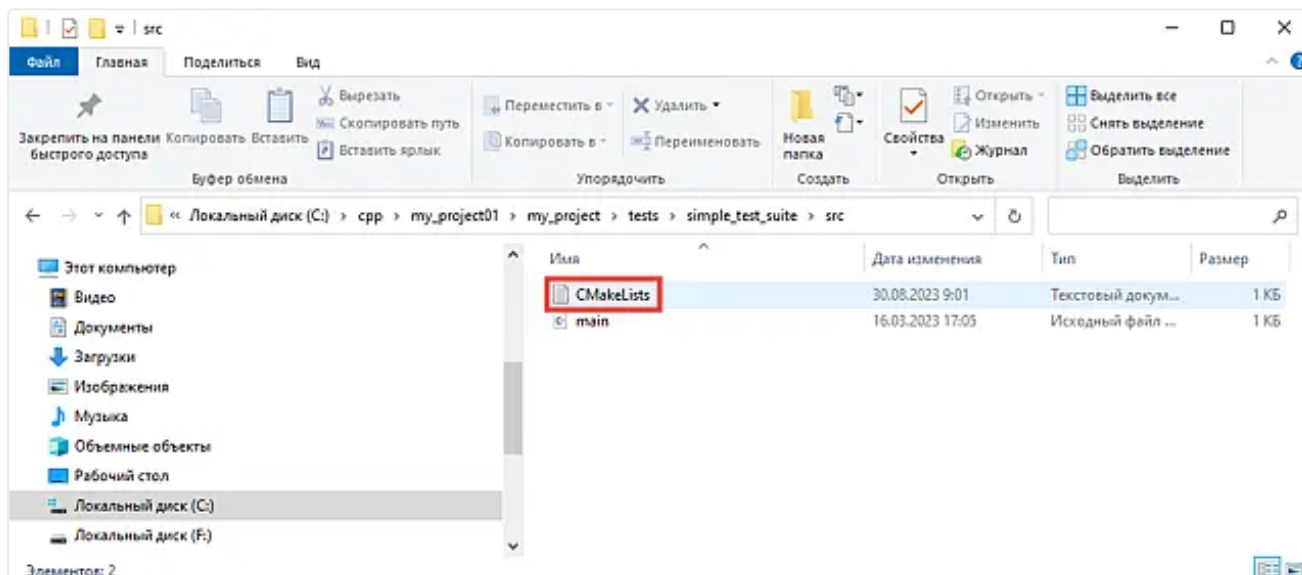
2. ninja - для сборки тестового проекта;

3. Тестовый фреймворк - для запуска тестов.

Перед запуском примеров

Все примеры тестов выполнены в [SBIS SDK](#) версии 23.5100.

Если вы установили SBIS SDK другой версии, поменяйте ее номер в файле CMakeLists.txt.



```
cmake_minimum_required(VERSION 3.5.2)
include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
sbis_create_project("test-some-lib" test DEPENDS
    boost_system
    boost_filesystem
    boost_unit_test_framework)
sbis_add_sdk_dependency( sbis-lib300 )
```

Для запуска тестов в docker-контейнере активируйте виртуальное окружение с помощью команды:

```
source /venv_conan/bin/activate (папка в корне)
```

Далее показан пример [простого теста на C++](#). Те же действия можно использовать для [простого теста на Python](#).

При разработке тестов для модулей БЛ скорее всего возникнет проблема "жирного облака", когда тестируемый модуль зависит от множества других напрямую или косвенно. В этом случае рекомендуется часть модулей заменить своими заглушками (пустой модуль с таким же uuid), часть функционала сторонних модулей перекрыть своими реализациями или мок-сервисом (когда нужно перекрыть именно удаленный вызов).

Простой тест

[Архив с файлами для запуска теста](#)

Данный тест рекомендуется всегда использовать при создании юнит-тестов.

Файловая структура каталога теста имеет следующий вид:

```
├─ my_project
│   └─ tests
│       └─ simple_test_suite
│           ├── src // каталог с исходниками тестов
│           │   ├── CMakeLists.txt // файл сборки теста
│           │   └─ main.cpp // исходный код теста
│           └─ test.entry // файл описания теста
```

В каталоге **src** расположены файл **main.cpp**, в котором размещен исходный код теста:

```
1 #include <sbis-lib/test.hpp>
2
```

```

3 BOOST_AUTO_TEST_SUITE( __main__ )
4
5 BOOST_AUTO_TEST_CASE( DummyTest )
6 {
7     BOOST_CHECK_EQUAL( 2 * 2, 4 );
8 }
9
10 BOOST_AUTO_TEST_SUITE_END()

```

и файл сборки теста CMakeLists.txt:

```

1 cmake_minimum_required(VERSION 3.5.2)
2 include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3 sbis_create_project("test-some-lib" test DEPENDS
4                     boost_system
5                     boost_filesystem
6                     boost_unit_test_framework)
7 sbis_add_sdk_dependency( sbis-lib300 )
8
9 target_sources(${target} PRIVATE
10    "main.cpp"
11 )
12
13 sbis_treat_warnings_as_errors()
14
15 sbis_enable_unity_build(20)
16
17 sbis_enable_static_build()

```

Описание теста в файле test.entry не требуется:

```

1 {
2 }

```

Для запуска теста выполните следующие шаги:

1. Создайте каталог **build** для сборки тестовых проектов. Размещение этого каталога определяется разработчиком. В нашем примере каталог создается на одном уровне с **my_project**.

```

├─ build
├─ my_project
│   └─ tests
│       └─ simple_test_suite
│           └─ src
│               ├── CMakeLists.txt
│               └─ main.cpp
│           └─ test.entry

```

2. Запустите тест:

- Откройте командную строку/консоль:
 - При работе в Windows откройте [Tensor C++ Developer Command Prompt 2025](#).
 - При работе в Linux откройте консоль и с помощью команды export [добавьте переменные окружения](#).
- Перейдите в каталог **build**.
- Выполните последовательно следующие команды:
 - Запуск генерации проекта.
 - Windows:

```

1 cmake ..\my_project\tests\simple_test_suite\src ^

```

```

2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-
  i686.cmake"

```

- Linux и macOS на Intel (x86_64):

```

1 cmake ../my_project/tests/simple_test_suite/src \
2     -DSBIS_UNIT_TESTS=1 -G Ninja

```

- macOS на Apple M1 (ARM):

```

1 cmake ../my_project/tests/simple_test_suite/src \
2     -DSBIS_ARCHITECTURE=arm64 \
3     -DSBIS_UNIT_TESTS=1 -G Ninja

```

- Запуск сборки проекта.

ninja

- Запуск тестирования проекта. Команды написаны для SBIS SDK 23.5100.

- Windows:

```

1 py -3.11-32 "%SBISPlatformSDK_235100%\tools\test_framework"
  "..\my_project\tests" ^
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"

```

- Linux:

```

1 /opt/python3.11/bin/python3.11
  $SBISPlatformSDK_235100/tools/test_framework
  ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee

```

- macOS:

```

1 python3.11 $SBISPlatformSDK_235100/tools/test_framework
  ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee

```

Простой тест с использованием ресурсов

 [Архив с файлами для запуска теста](#)

Файловая структура каталога теста имеет следующий вид:

```

├─ my_project
│   └─ tests
│       └─ res_test_suite
│           ├── resources // каталог для дополнительных файлов
│           │   └─ hello_world.txt // текстовый файл
│           ├── src // каталог с исходниками тестов
│           │   ├── CMakeLists.txt // файл сборки теста
│           │   └─ main.cpp // исходный код теста
│           └─ test.entry // файл описания теста

```

В каталог **resources** помещаются дополнительные файлы, на которые после запуска теста будут созданы символические ссылки (в данном примере - файл `hello_world.txt` с текстом "Hello, World!"). Также в каталоге может располагаться [файл `sbis-rpc-service.ini`](#) с параметрами тестового облака.

В каталоге **src** расположены файл `main.cpp`, в котором размещен исходный код теста:

```

1 #include <sbis-lib/test.hpp>

```

```

2  #include <fstream>
3
4  namespace sbis
5  {
6
7  BOOST_AUTO_TEST_SUITE( __main__ )
8
9  BOOST_AUTO_TEST_CASE( HelloWorldTestCase )
10 {
11     std::ifstream in( "hello_world.txt" );
12     std::string hello_world;
13     std::getline( in, hello_world );
14     BOOST_CHECK_EQUAL( StrDecode< Encoding::UTF8 >( hello_world ),
        L"Hello, World!" );
15 }
16
17 BOOST_AUTO_TEST_SUITE_END()
18
19 } // namespace sbis

```

и файл сборки теста CMakeLists.txt:

```

1  cmake_minimum_required(VERSION 3.5.2)
2  include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3  sbis_create_project("test-some-lib" test DEPENDS
4                      boost_system
5                      boost_filesystem
6                      boost_unit_test_framework)
7  sbis_add_sdk_dependency( sbis-lib300 )
8
9  target_sources(${target} PRIVATE
10     "main.cpp"
11 )
12
13 sbis_treat_warnings_as_errors()
14
15 sbis_enable_unity_build(20)
16
17 sbis_enable_static_build()

```

Описание теста в файле test.entry не требуется, т.к. на все файлы из каталога **resources** символические ссылки создаются автоматически:

```

1  {
2  }

```

Для запуска теста выполните следующие шаги:

1. Создайте каталог **build** для сборки тестовых проектов. Размещение этого каталога определяется разработчиком. В нашем примере каталог создается на одном уровне с **my_project**.

```

|— build
|— my_project
|   |— tests
|       |— res_test_suite
|           |— resources
|               |— hello_world.txt
|               |— src

```


- title – опциональный параметр, позволяет переопределить имя, под которым сайт будет зарегистрирован в конфигурационных файлах;

Каждое облако определяется в параметре services и разворачивается в подкаталоге каталога **run**. Если для работы сайта требуется БД, то в облаке должен быть определен шаблон развертывания БД.

Адрес развертываемого сервиса автоматически прописывается в файле sbis-rpc-service.ini в каталоге **run**. Если вам для работы сервиса необходимо задавать какие-либо произвольные параметры, то разместите файл sbis-rpc-service.ini в каталоге **/<корень источников теста>/clouds/<имя тестового облака>/**.

Пример.

Если у вас есть структура каталога, где лежит файл test_cloud.s3cld:

```

├─ my_project
│   └─ tests
│       └─ site_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   ├── SomeModule // каталог тестового модуля
│           │   │   ├── SomeModule.orx // справочник объектов
│           │   │   └─ SomeModule.s3mod // тестовый модуль
│           │   └─ test_cloud // каталог тестового приложения
│           │       ├── test_cloud.s3cld // тестовое облако
│           │       └─ test.s3srv // тестовый сервис
│           ├── resources // каталог для дополнительных файлов
│           └─ sbis-rpc-service.ini // файл с параметрами тестового
облака
│       ├── src // каталог с исходниками тестов
│       ├── CMakeLists.txt // файл сборки теста
│       ├── main.cpp // исходный код теста
│       └─ test.entry // файл описания теста

```

а файл test.entry имеет следующий вид:

```

1  {
2      "services" : [
3          {
4              "cloud" : "test_cloud"
5          }
6      ]
7  }

```

то при развороте будет сформирована структура каталогов:

```

1  └─ build
2      └─ tests
3          └─ SiteTestSuite
4              └─ run
5                  └─ sbis-rpc-service.ini
6  └─ my_project
7      └─ tests
8          └─ SiteTestSuite
9              ├── clouds
10                 ├── SomeModule
11                     ├── SomeModule.orx
12                     └─ SomeModule.s3mod
13                 └─ test_cloud
14                     ├── test_cloud.s3cld
15                     └─ test.s3srv
16                 ├── resources
17                 └─ sbis-rpc-service.ini

```

```

18 | | | src
19 | | | | CMakeLists.txt
20 | | | | main.cpp
21 | | | test.entry

```

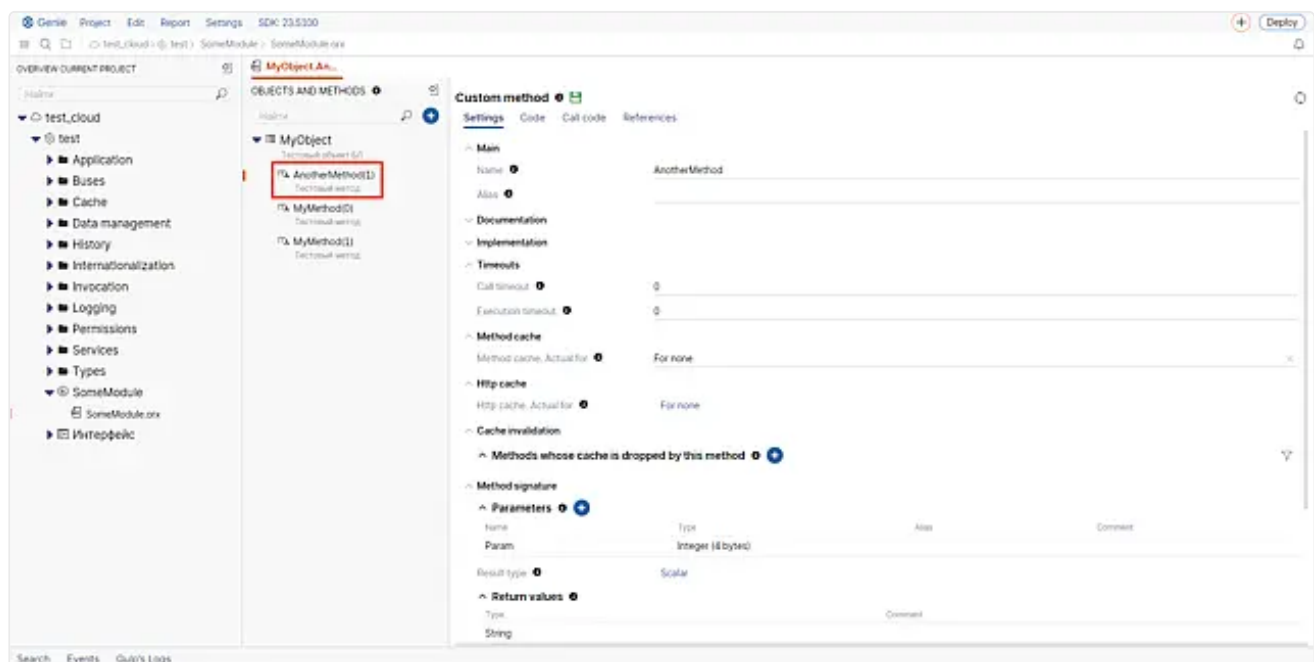
и в файл sbis-rpc-service.ini каталога **run** будут добавлены секции:

```

1  [test]
2  Адрес = http://r02-sizonenko1.region.tensor.ru:10000
3  БазаДанных = postgresql: host='localhost'
4                      port='5432'
5                      dbname='_sbis_test_db_2023-04-02T22-14-13-
6                      062939'
7                      user='postgres'
                        password='<пароль вашей локальной БД>'

```

Откройте проект test_cloud из каталога **clouds** с помощью [Genie](#). В модуле SomeModule в объекте MyObject добавлен метод AnotherMethod:



В каталоге **src** расположены файл main.cpp, в котором размещен исходный код теста:

```

1  #include <sbis-lib/test.hpp>
2  #include <sbis-bl-core/object.hpp>
3  #include <sbis-lib/types/record/record.hpp>
4
5  namespace sbis
6  {
7
8  BOOST_AUTO_TEST_SUITE( __main__ )
9
10 BOOST_AUTO_TEST_CASE( TextFrameworkExample )
11 {
12     blcore::Object obj( L"MyObject", L"test" );
13     String res;
14     BOOST_REQUIRE_NO_THROW( res = obj.Invoke< String >(
15         L"AnotherMethod", 333 ) );
16     BOOST_CHECK_EQUAL( res, L"SuperText" );
17 }

```



```

18 BOOST_AUTO_TEST_SUITE_END()
19
20 } // namespace sbis

```

и файл сборки теста CMakeLists.txt:

```

1 cmake_minimum_required(VERSION 3.5.2)
2 include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3 sbis_create_project("test-some-lib" test DEPENDS
4                     boost_system
5                     boost_filesystem
6                     boost_unit_test_framework)
7 sbis_add_sdk_dependencies( sbis-lib300 sbis-bl-core300 )
8
9 target_sources(${target} PRIVATE
10               "main.cpp"
11             )
12
13 sbis_treat_warnings_as_errors()
14
15 sbis_enable_unity_build(20)
16
17 sbis_enable_static_build()

```

Для запуска теста выполните следующие шаги:

1. Создайте каталог **build** для сборки тестовых проектов. Размещение этого каталога определяется разработчиком. В нашем примере каталог создается на одном уровне с **my_project**.

```

├─ build
├─ my_project
│   └─ tests
│       └─ site_test_suite
│           ├── clouds
│           │   ├── SomeModule
│           │   │   ├── SomeModule.orx
│           │   │   └─ SomeModule.s3mod
│           │   └─ test_cloud
│           │       ├── test_cloud.s3cld
│           │       └─ test.s3srv
│           ├── resources
│           │   └─ sbis-rpc-service.ini
│           ├── src
│           │   ├── CMakeLists.txt
│           │   └─ main.cpp
│           └─ test.entry

```

2. Запустите тест:

- Откройте командную строку/консоль:
 - При работе в Windows откройте [Tensor C++ Developer Command Prompt 2025](#).
 - При работе в Linux откройте консоль и с помощью команды export [добавьте переменные окружения](#).
- Перейдите в каталог **build**.
- Выполните последовательно следующие команды:
 - Запуск генерации проекта.

- Windows

```

1 cmake ..\my_project\tests\site_test_suite\src ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^

```

```
3 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-  
i686.cmake"
```

- Linux и macOS на Intel (x86_64):

```
1 cmake ../my_project/tests/site_test_suite/src \  
2 -DSBIS_UNIT_TESTS=1 -G Ninja
```

- macOS на Apple M1 (ARM):

```
1 cmake ../my_project/tests/site_test_suite/src \  
2 -DSBIS_ARCHITECTURE=arm64 \  
3 -DSBIS_UNIT_TESTS=1 -G Ninja
```

- Запуск сборки проекта.

```
ninja
```

- Запуск тестирования проекта. Команды написаны для SBIS SDK 23.5100.

- Windows

```
1 py -3.11-32 "%SBISPlatformSDK_235100%\tools\test_framework"  
   "..\my_project\tests" ^  
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"
```

- Linux

```
1 /opt/python3.11/bin/python3.11  
  $SBISPlatformSDK_235100/tools/test_framework  
  ../my_project/tests \  
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

- macOS

```
1 python3.11 $SBISPlatformSDK_235100/tools/test_framework  
  ../my_project/tests \  
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

Тест с использованием Redis



[Архив с файлами для запуска теста](#)

Файловая структура каталога теста имеет следующий вид:

```
|— my_project  
|   |— tests  
|       |— redis_test_suite  
|           |— src // каталог с исходниками тестов  
|               |— CMakeLists.txt // файл сборки теста  
|               |— main.cpp // исходный код теста  
|               |— test.entry // файл описания теста
```

В каталоге **src** расположены файл **main.cpp**, в котором размещен исходный код теста

```
1 #include <sbis-lib/test.hpp>  
2 #include <sbis-redis-client/client.hpp>  
3 namespace sbis{  
4  
5 BOOST_AUTO_TEST_SUITE( __main__ )  
6  
7 BOOST_AUTO_TEST_CASE( DummyTest )
```

```

8  {
9      redis::Client redisClient( L"Тест.Хранилище", true );
10     String key = L"123";
11     bool key_exists = redisClient.IsExists( key );
12     BOOST_CHECK_EQUAL( key_exists, false );
13     redisClient.SetValue( key, L"Value" );
14     String value;
15     redisClient.Value( value, key );
16     BOOST_CHECK_EQUAL( value, L"Value" );
17 }
18
19 BOOST_AUTO_TEST_SUITE_END()
20
21 }

```

и файл сборки теста CMakeLists.txt:

```

1  cmake_minimum_required(VERSION 3.5.2)
2  include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3  sbis_create_project("test-some-lib" test DEPENDS
4                      boost_system
5                      boost_filesystem
6                      boost_unit_test_framework)
7  sbis_add_sdk_dependencies( sbis-lib300 sbis-redis-client300 )
8
9  target_sources(${target} PRIVATE
10     "main.cpp"
11 )
12
13 sbis_treat_warnings_as_errors()
14
15 sbis_enable_unity_build(20)
16
17 sbis_enable_static_build()

```

Для того чтобы использовать Redis в юнит-тестах, укажите в файле test.entry имя хранилища Redis:

```

1  {
2      "redis" : [ "Тест.Хранилище" ]
3  }

```

Для запуска теста выполните следующие шаги:

1. Создайте каталог **build** для сборки тестовых проектов. Размещение этого каталога определяется разработчиком. В нашем примере каталог создается на одном уровне с **my_project**.

```

|— build
|— my_project
|   |— tests
|       |— redis_test_suite
|           |— resources
|               |— hello_world.txt
|           |— src
|               |— CMakeLists.txt
|               |— main.cpp
|           |— test.entry

```

2. Запустите тест:

- Откройте командную строку/консоль:

- При работе в Windows откройте [Tensor C++ Developer Command Prompt 2025](#).
- При работе в Linux откройте консоль и с помощью команды export [добавьте переменные окружения](#).
- Перейдите в каталог **build**.
- Выполните последовательно следующие команды:
 - Запуск генерации проекта.
 - Windows


```
1 cmake ..\my_project\tests\redis_test_suite\src ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-
   i686.cmake"
```
 - Linux и macOS на Intel (x86_64):


```
cmake ..\my_project\tests\redis_test_suite\src -DSBIS_UNIT_TESTS=1
-G Ninja
```
 - macOS на Apple M1 (ARM):


```
cmake ../my_project/tests/redis_test_suite/src -
DSBIS_ARCHITECTURE=arm64 -DSBIS_UNIT_TESTS=1 -G Ninja
```
 - Запуск сборки проекта.


```
ninja
```
 - Запуск тестирования проекта. Команды написаны для SBIS SDK 23.5100.
 - Windows


```
1 py -3.11-32 "%SBISPlatformSDK_235100%\tools\test_framework"
   "..\my_project\tests" ^
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"
```
 - Linux


```
1 /opt/python3.11/bin/python3.11
   $SBISPlatformSDK_235100/tools/test_framework
   ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```
 - macOS


```
1 python3.11 $SBISPlatformSDK_235100/tools/test_framework
   ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

Тест с использованием БД



[Архив с файлами для запуска теста](#)

Файловая структура каталога теста имеет следующий вид:

```

├─ my_project
│   └─ tests
│       └─ database_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   └─ SomeModule // каталог тестового модуля
│           │       ├── SomeModule.dicx // словарь данных
│           │       └─ SomeModule.dpack // пакет данных

```

```

├── SomeModule.s3mod // файл тестового модуля
├── test_cloud // каталог тестового приложения
│   ├── test_cloud.s3cld // файл тестового облака
│   └── test.s3srv // тестовый сервис
├── resources // каталог для дополнительных файлов
│   └── sbis-rpc-service.ini // файл с параметрами тестового
облака
├── src // каталог с исходниками тестов
│   ├── CMakeLists.txt // файл сборки теста
│   ├── main.cpp // исходный код
└── test.entry // файл описания теста

```

Базы данных разворачиваются из s3cld-файлов в каталоге **clouds** и задаются в файле test.entry параметром **"databases"**. Значением параметра является массив словарей с элементами:

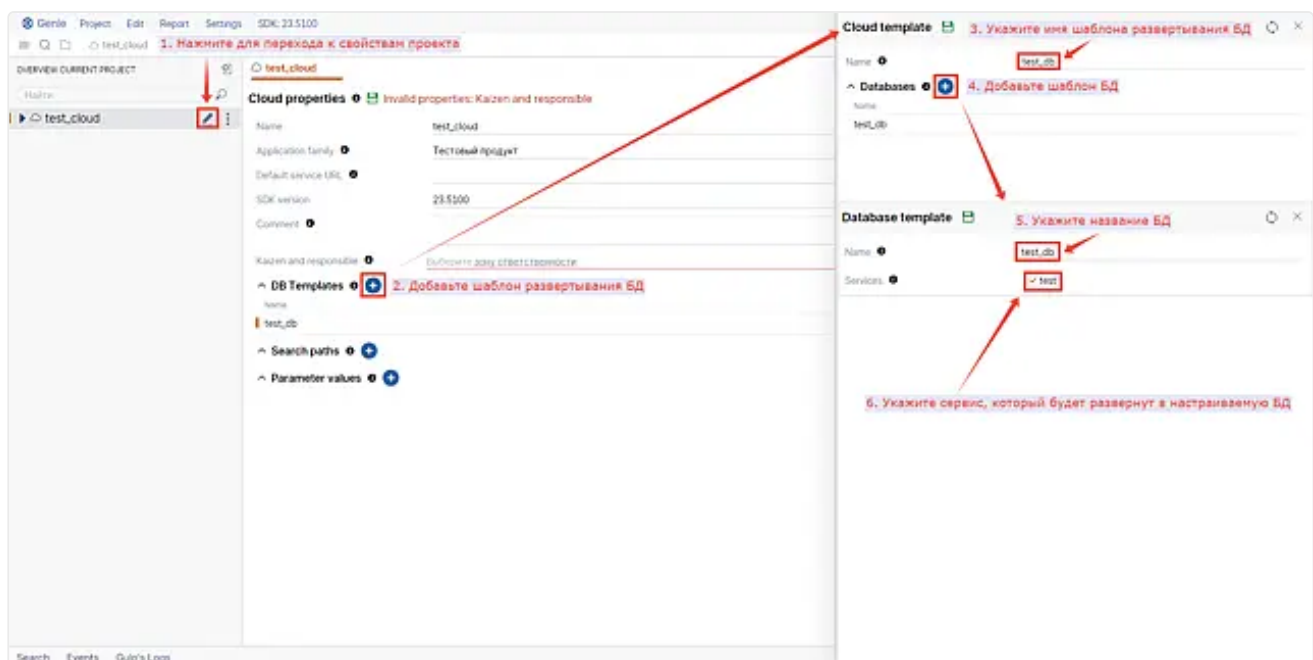
- **"cloud"** – наименование облака, по которому необходимо развернуть БД;
- **"param"** – имя параметра в конфигурационном файле, с которым будет связана строка подключения к БД.



Предупреждение

Использование кириллицы в путях к файлам запрещено.

В одном s3cld-файле должен быть ровно один шаблон разворачивания БД. В настройках проекта в опции **DB Templates** указывается имя шаблона разворачивания БД.



Помимо описания БД в вашем сервисе могут храниться файлы пакета данных для первичного наполнения разворачивания БД.

Файл тестового облака test_cloud открывается в [Genie](#). В настройках проекта в параметре **DB Templates** создана схема разворачивания БД test_db.

В облаке проекта test_cloud создается тестовый модуль SomeModule.s3mod, который содержит:

- [Словарь данных](#) SomeModule с таблицей ТестоваяТаблица.

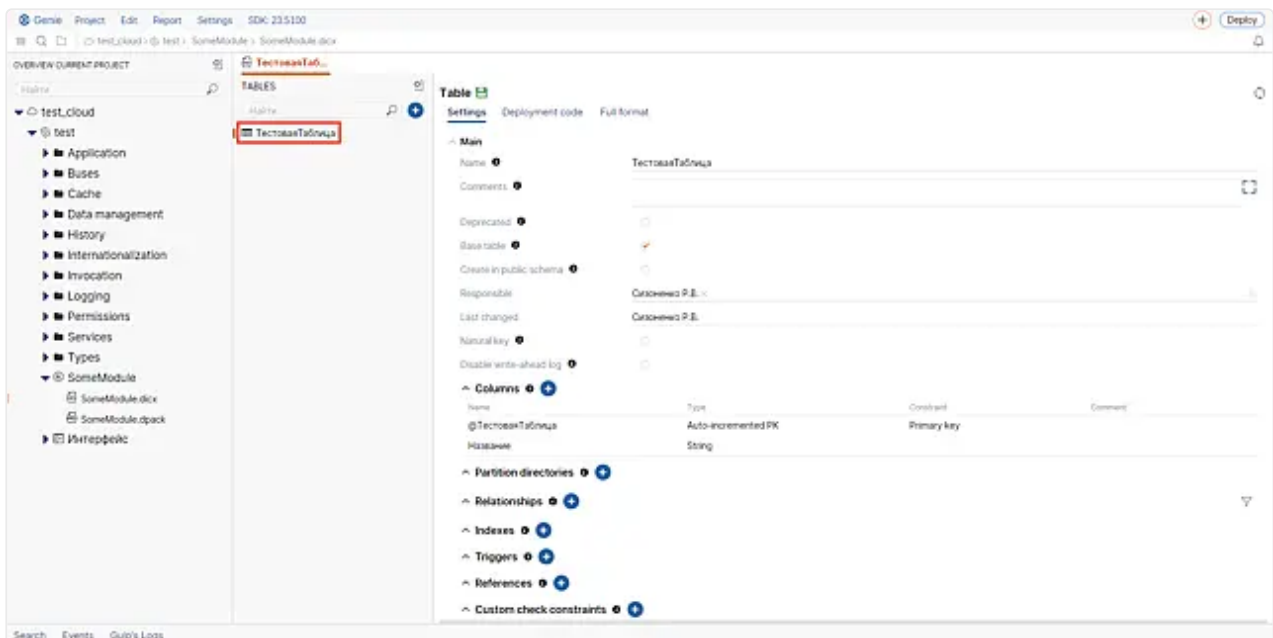
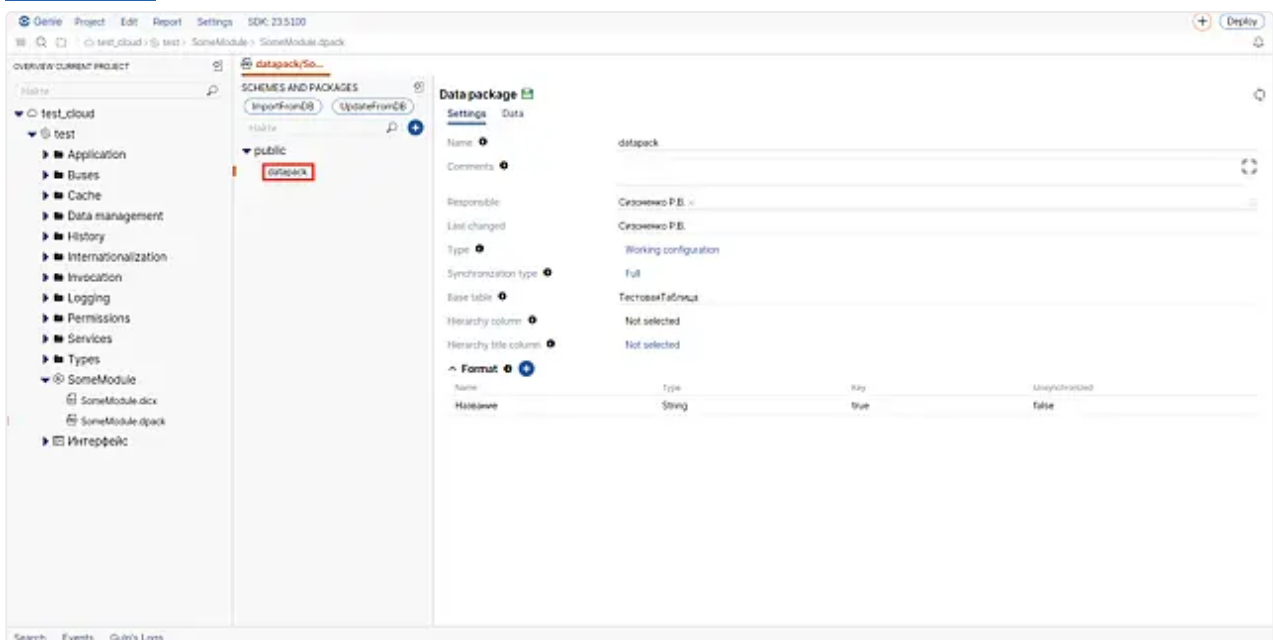
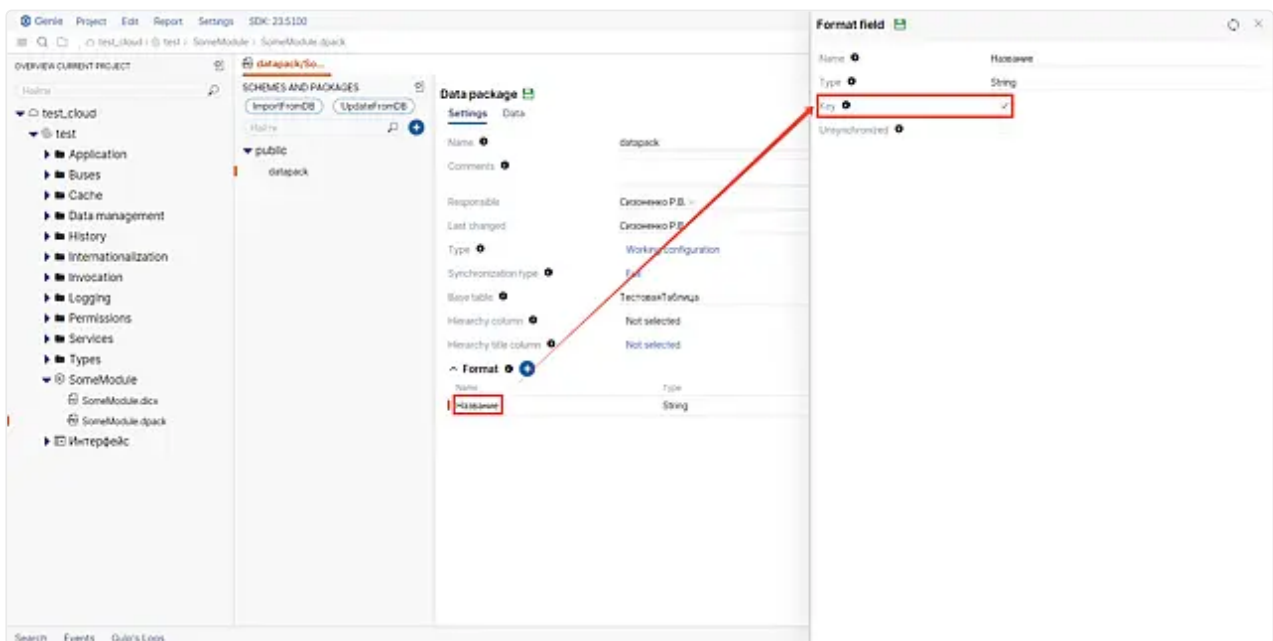


Таблица содержит колонку с именем Название и типом данных String.

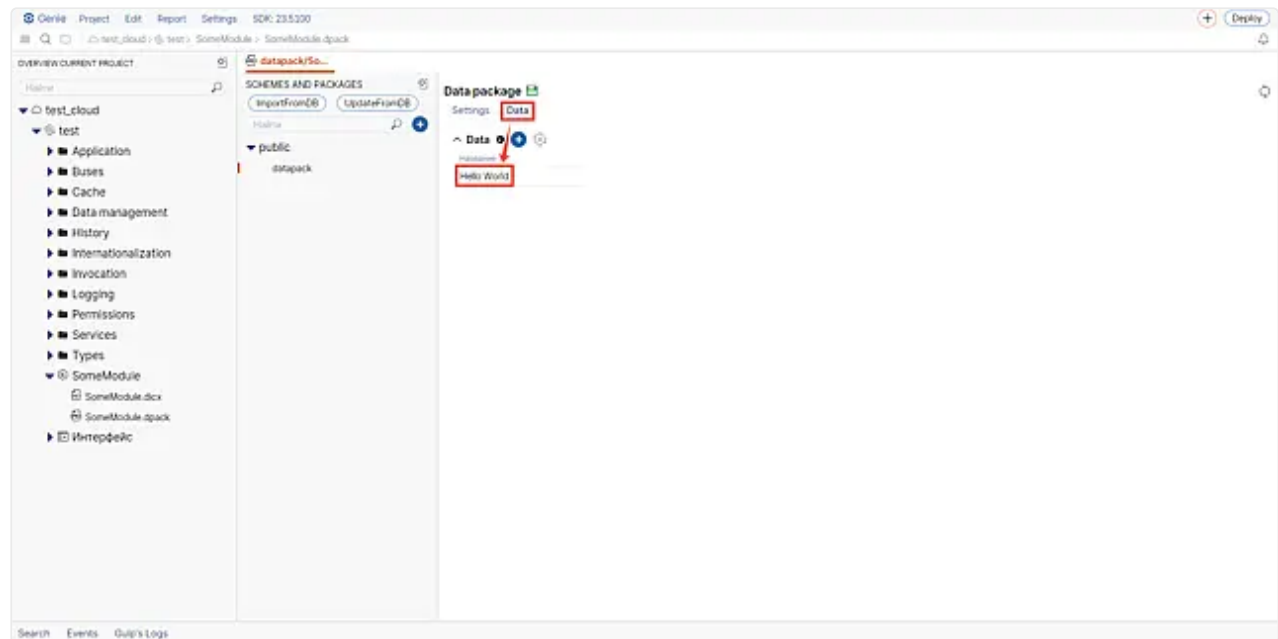
- [Пакет данных](#) SomeModule с пакетом datapack.



Тип синхронизации Full, а в опции **Initialize format from table** указана таблица ТестоваяТаблица. В опции **Format** для колонки Название указан флаг Key.



На вкладке пакета данных **Data** добавлено текстовое значение Hello World:



В файл описания теста test.entry добавлен параметр databases:

```
1 {
2     "databases": [
3         {
4             "cloud" : "test_cloud",
5             "param" : "БазаДанных"
6         }
7     ]
8 }
```

В каталоге **src** расположены файл main.cpp, в котором размещен исходный код теста:

```
1 #include <sbis-lib/test.hpp>
2 #include <sbis-bl-core/object.hpp>
3 #include <sbis-lib/types/record/record.hpp>
4 #include <sbis-lib/test.hpp>
5 #include <sbis-dbi/dbi/idatabase.hpp>
6 #include <sbis-dbi/dbi/istatement.hpp>
7 #include <sbis-dbi/dbi/connection_params.hpp>
8 #include <sbis-lib/utils/config.hpp>
9 #include <sbis-dbi-postgres/static_driver_loader.hpp>
10 #include <sbis-db-manager/db_manager.hpp>
11
12 namespace sbis
13 {
14
15 BOOST_AUTO_TEST_SUITE( __main__ )
16
17 BOOST_AUTO_TEST_CASE( ReadFromDb1TestCase )
18 {
19     // открываем соединение с БД
20     auto db = db_manager::GetConnectedDataDB( );
21     auto stmt = db->CreateStatement();
22     std::shared_ptr< ICursor > result;
23     // получаем данных из таблицы в созданном пакете данных
24     BOOST_REQUIRE_NO_THROW( result = stmt->Exec( L"SELECT \"Название\"
FROM \"ТестоваяТаблица\"" ) );
```

```

25     BOOST_REQUIRE( result->Next() );
26     Record const& data = result->Data();
27     BOOST_REQUIRE_EQUAL( data.Count(), 1u );
28     // проверяем, что в БД лежит одна строка "Hello World"
29     BOOST_CHECK_EQUAL( data[ 0 ].Ref< String >(), L"Hello World" );
30     BOOST_REQUIRE( !result->Next() );
31 }
32
33 BOOST_AUTO_TEST_SUITE_END()
34
35 static dbi::postgres::StaticDriverLoader REG;
36
37 } // namespace sbis

```

и файл сборки теста CMakeLists.txt:

```

1  cmake_minimum_required(VERSION 3.5.2)
2  include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3  sbis_create_project("test-some-lib" test DEPENDS
4                      boost_system
5                      boost_filesystem
6                      boost_unit_test_framework)
7  sbis_add_sdk_dependencies( sbis-lib300 sbis-bl-core300 sbis-lang300
8                             sbis-rpc300 sbis-xml300 sbis-xerces300 sbis-dbi300 sbis-dbi-postgres
9                             sbis-dbi-sqlite sbis-bl-metadata-basic300 sbis-dbi-postgres sbis-db-
10                             manager300)
11
12  target_sources(${target} PRIVATE
13                 "main.cpp"
14 )
15
16  sbis_treat_warnings_as_errors()
17
18  sbis_enable_unity_build(20)
19
20  sbis_enable_static_build()

```

Для запуска теста выполните следующие шаги:

1. Создайте каталог **build** для сборки тестовых проектов. Размещение этого каталога определяется разработчиком. В нашем примере каталог создается на одном уровне с **my_project**.

```

├─ build
├─ my_project
│   └─ tests
│       └─ database_test_suite
│           ├── clouds
│           │   ├── SomeModule
│           │   │   ├── SomeModule.dicx
│           │   │   ├── SomeModule.dpack
│           │   │   └─ SomeModule.s3mod
│           │   └─ test_cloud
│           │       ├── test_cloud.s3cld
│           │       └─ test.s3srv
│           ├── resources
│           │   └─ sbis-rpc-service.ini
│           ├── src
│           └─ CMakeLists.txt

```



```

|
|
|   └─ main.cpp
|   └─ test.entry

```

2. Запустите тест:

- Откройте командную строку/консоль:
 - При работе в Windows откройте [Tensor C++ Developer Command Prompt 2025](#).
 - При работе в Linux откройте консоль и с помощью команды export [добавьте переменные окружения](#).
- Перейдите в каталог **build**.
- Выполните последовательно следующие команды:

■ Windows

```

1 cmake ..\my_project\tests\database_test_suite\src ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-
  i686.cmake"

```

■ Linux и macOS на Intel (x86_64):

```

1 cmake ../my_project/tests/database_test_suite/src \
2 -DSBIS_UNIT_TESTS=1 -G Ninja

```

■ macOS на Apple M1 (ARM):

```

1 cmake ../my_project/tests/database_test_suite/src \
2 -DSBIS_ARCHITECTURE=arm64 \
3 -DSBIS_UNIT_TESTS=1 -G Ninja

```

- Запуск сборки проекта.

```
ninja
```

- Запуск тестирования проекта. Команды написаны для SBIS SDK 23.5100.

■ Windows

```

1 py -3.11-32 "%SBISPlatformSDK_235100%\tools\test_framework"
  "..\my_project\tests" ^
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"

```

■ Linux

```

1 /opt/python3.11/bin/python3.11
  $SBISPlatformSDK_235100/tools/test_framework
  ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee

```

■ macOS

```

1 python3.11 $SBISPlatformSDK_235100/tools/test_framework
  ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee

```

Тест мобильных микросервисов



[Архив с файлами для запуска теста](#)

Основное отличие тестов мобильных микросервисов от остальных заключается в том, что у мобильных микросервисов гораздо больше требований к окружению. Для соответствия этим требованиям необходимо запускать целый сервис, поэтому тесты нельзя располагать в исполняемом файле и создается модуль.

Чтобы этот модуль попал в запускаемый тестируемый сервис нужен модуль в терминах s3mod, у которого указывается библиотека.

Типовая структура каталогов микросервиса

Микросервис, для которого необходимо добавить тесты, описан в файлах `my_microservice.s3cld` и `my_microservice.s3srv` каталога **projects**:

```
├─ my_project
│   └─ projects
│       └─ my_microservice // каталог микросервиса
│           ├── my_microservice.s3srv // файл проекта
│           └─ my_microservice.s3cld // файл сервиса
```

и содержит модуль `Module.s3mod`, расположенный в каталоге **www/service/Module**:

```
├─ www
│   └─ service
│       └─ Module // каталог модуля
│           └─ Module.s3mod // файл модуля микросервиса
```

Исходные файлы модуля хранятся в каталоге **src**, который расположен в корне каталога теста на одном уровне с **projects** и **www**:

```
├─ my_project
│   ├── projects
│   │   └─ my_microservice
│   │       ├── my_microservice.s3srv
│   │       └─ my_microservice.s3cld
│   ├── src // каталог исходных файлов модуля
│   │   └─ module
│   │       ├── implementation
│   │       │   └─ source.cpp // исходный код
│   │       ├── module
│   │       │   └─ source.hpp // заголовочный файл
│   │       └─ CMakeLists.txt // файл сборки исходников модуля
│   ├── www
│   │   └─ service
│   │       └─ Module
│   │           └─ Module.s3mod
│   └─ CMakeLists.txt // файл сборки для подключения теста и тестового модуля
```

В корне каталога проекта размещается файл сборки `CMakeLists.txt`, который подключает тест и тестовый модуль:

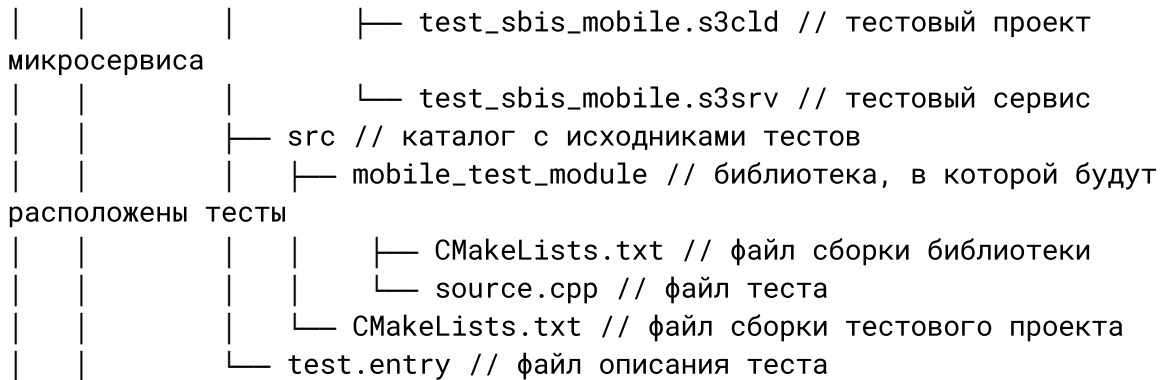
```
1 cmake_minimum_required(VERSION 3.5.2)
2 include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3
4 add_subdirectory( src/module )
```

Добавление тестов для микросервиса

Файловая структура тестового каталога имеет следующий вид:

```
├─ my_project
│   └─ tests
│       └─ test_my_microservice
│           ├── clouds // каталог для метаданных проекта
│           │   └─ test_sbis_mobile // каталог тестового приложения
│           │       ├── TestModule // каталог тестового модуля
│           │       │   └─ TestModule.s3mod // модуль библиотеки с тестами
│           │       └─ sbis-rpc-service.ini // файл с параметрами тестового
```

облака



В сервисе test_sbis_mobile.s3srv будут запускаться тесты. Каталог **src** содержит два файла CMakeLists.txt:

1. Отвечает за исполняемый файл тестов, т.е. то, что будет запускаться тестовым фреймворком:

```
1 cmake_minimum_required(VERSION 3.5.2)
2 include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake"
3 )
4 sbis_create_project( "test_my_microservice" test_mobile DEPENDS
5 boost_system boost_filesystem boost_unit_test_framework) // описание
6 проекта
7
8 sbis_add_sdk_dependencies(
9     sbis-lib300
10    sbis-loader300
11    sbis-rpc-service300
12 )
13
14 sbis_enable_unity_build(20)
15
16 sbis_apply_settings()
17
18 add_subdirectory( mobile_test_module ) // подключение библиотеки, где
19 лежат тесты
```

2. Отвечает за библиотеку, в которой будут располагаться тесты:

```
1 sbis_create_project( "mobile_test_module" library DEPENDS
2 boost_filesystem boost_system boost_thread boost_unit_test_framework)
3 // описание библиотеки
4
5 sbis_add_sdk_dependencies(
6     sbis-lib300
7 )
8
9 sbis_add_dependencies(
10    module
11 )
12
13 target_sources(${target} PRIVATE
14    "source.cpp"
15 )
16
17 sbis_enable_unity_build(20)
18
19 sbis_apply_settings()
```

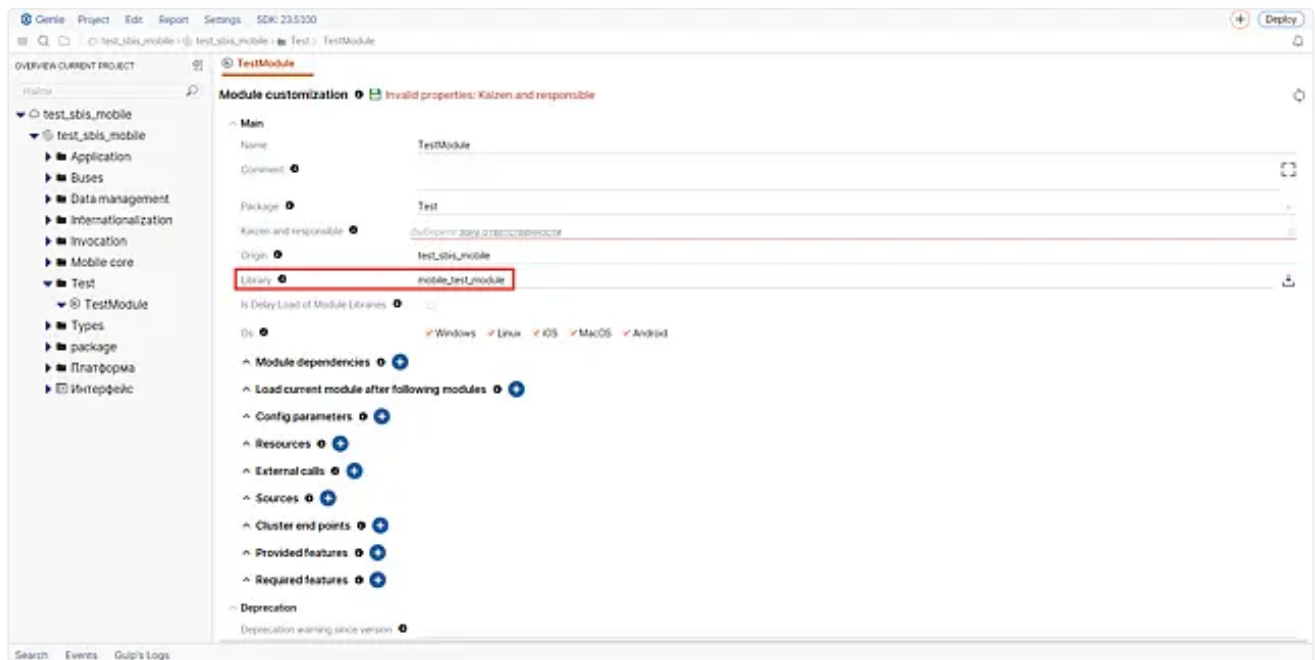
и файл теста source.cpp:

```

1  #include <sbis-lib/test.hpp>
2  #include <module/source.hpp>
3
4  BOOST_AUTO_TEST_SUITE( __main__ )
5
6  BOOST_AUTO_TEST_CASE( DummyTest )
7
8  {
9      BOOST_CHECK_EQUAL( module_test::GetValue(), 10 );
10 }
11
12 BOOST_AUTO_TEST_SUITE_END()

```

Библиотека с тестами `mobile_test_module` указывается для модуля `TestModule.s3mod`, который будет поставлять тесты сервису `test_sbis_mobile.s3srv`.



Файл описания теста `test.entry` включает в себя параметры:

- `mobile_controller` - тест мобильного контроллера с именем `s3cid`-файла `test_sbis_mobile`, который должен быть развернут.
- `project_under_test` - содержит ссылку на оригинальный файл микросервиса `my_microservice.s3srv`.

```

1  {
2      "mobile_controller": "test_sbis_mobile",
3      "project_under_test":
4      "../../projects/my_microservice/my_microservice.s3srv"
5  }

```

Добавьте тестовый каталог в файл `CMakeLists.txt`, расположенный в корне каталога проекта:

```

1  cmake_minimum_required(VERSION 3.5.2)
2  include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3
4  add_subdirectory( src/module )
5
6  add_subdirectory( tests/test_my_microservice/src )

```

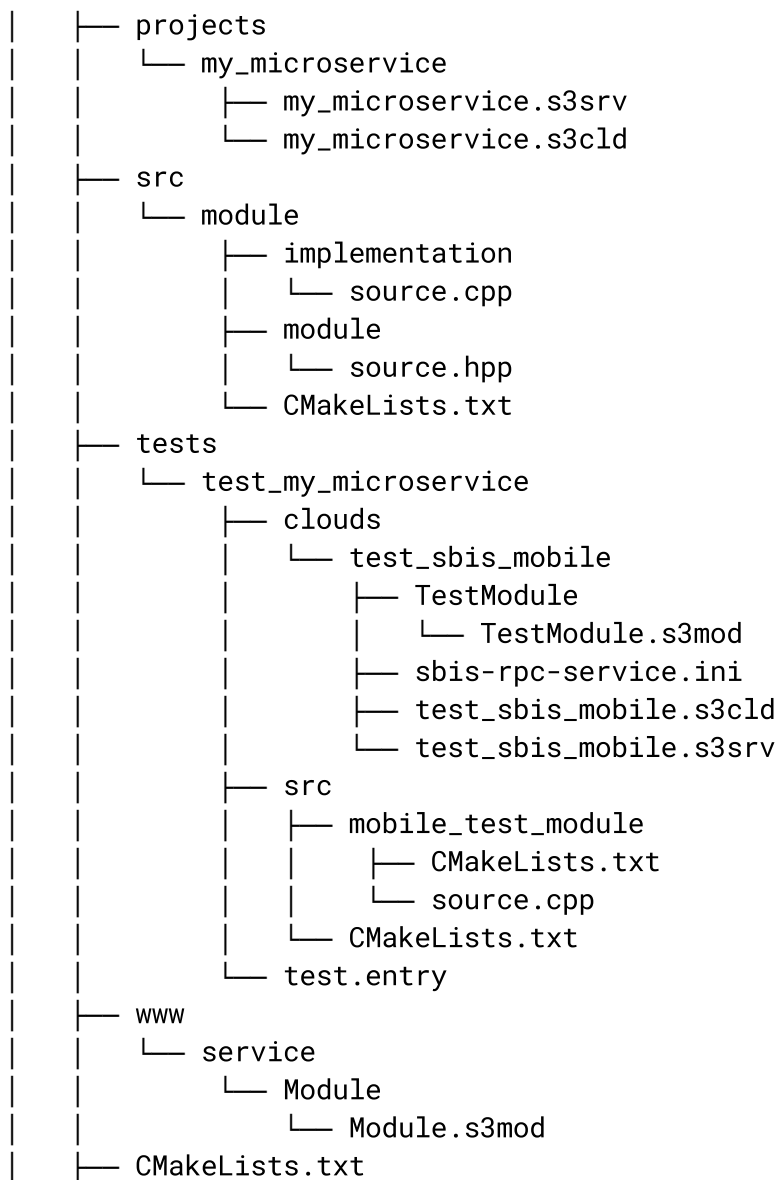
Для запуска теста выполните следующие шаги:

1. Создайте каталог **build** для сборки тестовых проектов. Размещение этого каталога определяется разработчиком. В нашем примере каталог создается на одном уровне с **my_project**.

```

├─ build
├─ my_project

```



2. Запустите тест:

- Откройте командную строку/консоль:
 - При работе в Windows откройте [Tensor C++ Developer Command Prompt 2025](#).
 - При работе в Linux откройте консоль и с помощью команды export [добавьте переменные окружения](#).
- Перейдите в каталог **build**.
- Выполните последовательно следующие команды:
 - Запуск генерации проекта.
 - Windows


```

1 cmake ..\my_project ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-i686.cmake"
```
 - Linux и macOS на Intel (x86_64):


```

1 cmake ..\my_project \
2 -DSBIS_UNIT_TESTS=1 -G Ninja
```
 - macOS на Apple M1 (ARM):


```

1 cmake ../my_project \
2 -DSBIS_ARCHITECTURE=arm64 \
3 -DSBIS_UNIT_TESTS=1 -G Ninja
```
 - Запуск сборки проекта.

- Запуск тестирования проекта. Команды написаны для SBIS SDK 23.5100.

- Windows

```
1 py -3.11-32 "%SBISPlatformSDK_235100%\tools\test_framework"
  "..\my_project\tests" ^
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"
```

- Linux

```
1 /opt/python3.11/bin/python3.11
  $SBISPlatformSDK_235100/tools/test_framework
  ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

- macOS

```
1 python3.11 $SBISPlatformSDK_235100/tools/test_framework
  ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

[Deprecated] Тест с использованием mock-сервиса



Данный способ устарел!

[Актуальное описание теста с использованием mock-сервиса](#)

Файловая структура каталога теста имеет следующий вид:

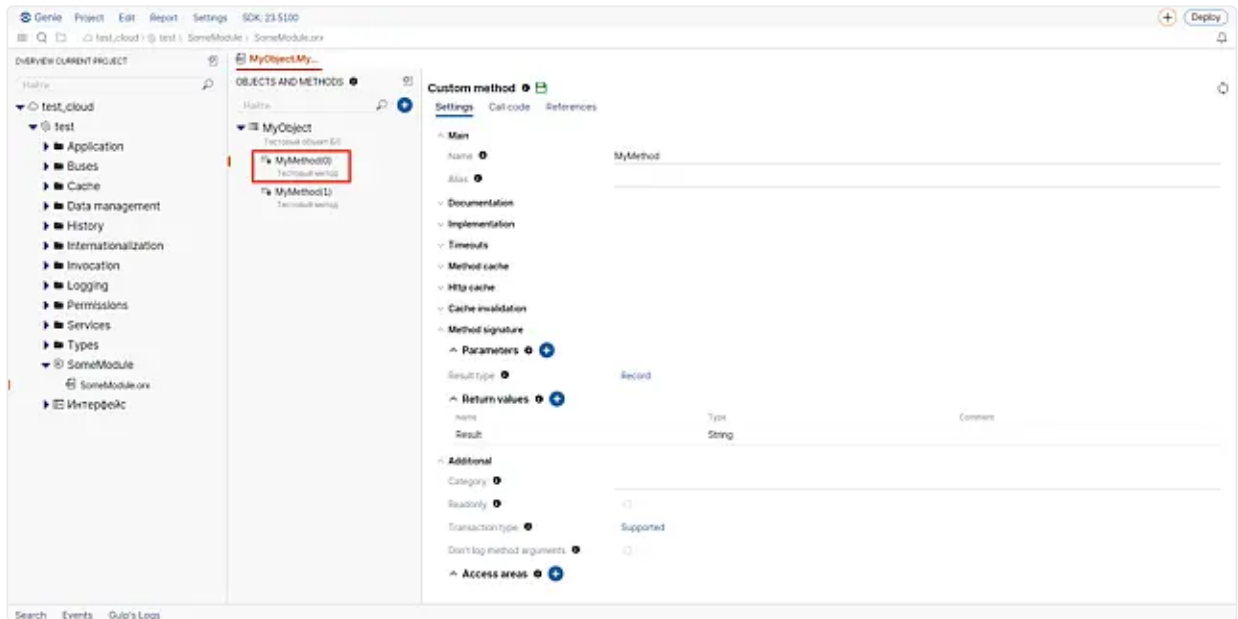
```
├─ my_project
│   └─ tests
│       └─ mock_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   ├── SomeModule // каталог тестового модуля
│           │   │   ├── SomeModule.orx // справочник объектов
│           │   │   └─ SomeModule.s3mod // тестовый модуль
│           │   └─ test_cloud // каталог тестового приложения
│           │       ├── test_cloud.s3cld // тестовое облако
│           │       └─ test.s3srv // тестовый сервис
│           ├── resources // каталог для дополнительных файлов
│           └─ sbis-rpc-service.ini // файл с параметрами тестового
облака
│       ├── src // каталог с исходниками тестов
│       │   ├── CMakeLists.txt // файл сборки теста
│       │   └─ main.cpp // исходный код теста
│       └─ test.entry // файл описания теста
```

В каталоге **clouds** располагаются метаданные проекта - файл облака test_cloud.s3cld и тестовый сервис test.s3srv.

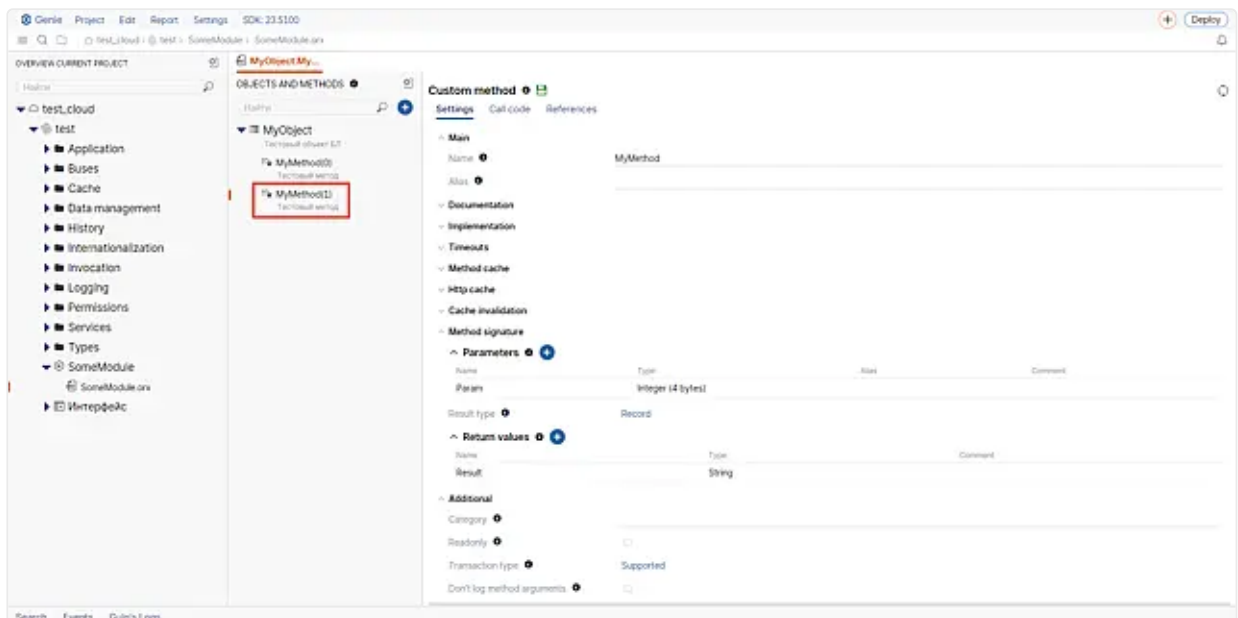
С помощью [Genie](#) откройте проект test_cloud.s3cld, в котором содержится модуль SomeModule со [справочником объектов](#) SomeModule.orx. Справочник содержит объект БЛ с именем MyObject и несколькими [произвольными методами](#).

- Первый метод:
 - тип Custom method;
 - атрибут Name — MyMethod;
 - атрибут Accessibility — Internal;
 - атрибут Method type — Native function;

- о без параметров;
- о атрибут Result type — Record;
- о атрибут Return values — текстовое поле Result (String).



- Второй метод:
 - о тип Custom method;
 - о атрибут Name — MyMethod;
 - о атрибут Accessibility — Internal;
 - о атрибут Method type — Native function;
 - о атрибут Parameters — целочисленное поле Param (Integer (4 bytes));
 - о атрибут Result type — Record;
 - о атрибут Return values — текстовое поле Result (String).



Каталог **resources** содержит конфигурационный файл **sbis-rpc-service.ini** с параметрами:

- 1 **[Ядро.Логирование]**
- 2 Уровень = Отладочный

Файл **test.entry** имеет следующий вид:

```

1 {
2     "mockservice" : {

```

```

3         "Тестовый сервис" : {
4             "address" : "/test_service",
5             "methods" : [
6                 [ "MyObject.MyMethod", { "s": [{ "n": "Result", "t":
"Строка" } ] },
7                 "d": [ "Это общий результат" ], "_type": "record" } ],
8                 [ "MyObject.MyMethod", 1, { "s": [{ "n": "Result", "t":
"Строка" } ] },
9                 "d": [ "Я возвращаю это только при 1 параметре" ],
10                "_type": "record" } ],
11                [ "MyObject.MyMethod", { "Param": 123 }, { "s": [{ "n":
"Result", "t": "Строка" } ] },
12                "d": [ "Я возвращаю это только для конкретных
параметров" ], "_type": "record" } ]
13            ],
14            "method_formats" : [
15                "clouds/test_cloud/SomeModule/SomeModule.orx"
16            ]
17        }
18    }

```

В параметре method_formats прописывается путь до справочника объектов SomeModule.orx.

В каталоге **src** расположены файл main.cpp, в котором размещен исходный код теста:

```

1  #include <sbis-lib/test.hpp>
2  #include <sbis-bl-core/object.hpp>
3  #include <sbis-lib/types/record/record.hpp>
4
5  namespace sbis
6  {
7
8  BOOST_AUTO_TEST_SUITE( __main__ )
9
10 BOOST_AUTO_TEST_CASE( MockWithoutArgsTestCase )
11 {
12     blcore::Object obj( L"MyObject", L"Тестовый сервис" );
13     Record res;
14     BOOST_REQUIRE_NO_THROW( res = obj.Invoke< Record >( L"MyMethod" ) );
15     BOOST_CHECK( res[ L"Result" ].To< String >() == L"Это общий
результат" );
16 }
17
18 BOOST_AUTO_TEST_CASE( MockOneArgTestCase )
19 {
20     blcore::Object obj( L"MyObject", L"Тестовый сервис" );
21     Record res;
22     BOOST_REQUIRE_NO_THROW( res = obj.Invoke< Record >( L"MyMethod", 222
) );
23     BOOST_CHECK( res[ L"Result" ].To< String >() == L"Я возвращаю это
только при 1 параметре" );
24 }
25
26 BOOST_AUTO_TEST_CASE( MockCustomArgCase )
27 {

```



```

28     blcore::Object obj( L"MyObject", L"Тестовый сервис" );
29     Record res;
30     BOOST_REQUIRE_NO_THROW( res = obj.Invoke< Record >( L"MyMethod", 123
) );
31     BOOST_CHECK( res[ L"Result" ].To< String >() == L"Я возвращаю это
только для конкретных параметров" );
32 }
33
34 BOOST_AUTO_TEST_SUITE_END()
35
36 } // namespace sbis

```

и файл сборки теста CMakeLists.txt:

```

1  cmake_minimum_required(VERSION 3.5.2)
2  include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3  sbis_create_project("test-some-lib" test DEPENDS
4                      boost_system
5                      boost_filesystem
6                      boost_unit_test_framework)
7  sbis_add_sdk_dependencies( sbis-lib300 sbis-bl-core300 sbis-lang300
sbis-rpc300 sbis-xml300 sbis-xerces300 sbis-dbi300 sbis-dbi-postgres
sbis-dbi-sqlite sbis-bl-metadata-basic300 sbis-dbi-postgres)
8
9  target_sources(${target} PRIVATE
10     "main.cpp"
11 )
12
13 sbis_treat_warnings_as_errors()
14
15 sbis_enable_unity_build(20)
16
17 sbis_enable_static_build()

```

Для запуска теста выполните следующие шаги:

1. Создайте каталог **build** для сборки тестовых проектов. Размещение этого каталога определяется разработчиком. В нашем примере каталог создается на одном уровне с **my_project**.

```

├─ build
├─ my_project
│   └─ tests
│       └─ mock_test_suite
│           ├── clouds
│           │   ├── SomeModule
│           │   │   ├── SomeModule.orx
│           │   │   └─ SomeModule.s3mod
│           │   └─ test_cloud
│           │       ├── test_cloud.s3cld
│           │       └─ test.s3srv
│           ├── resources
│           │   └─ sbis-rpc-service.ini
│           ├── src
│           │   ├── CMakeLists.txt
│           │   └─ main.cpp
│           └─ test.entry

```

2. Запустите тест:

- Откройте командную строку/консоль:
 - При работе в Windows откройте [Tensor C++ Developer Command Prompt 2025](#).
 - При работе в Linux откройте консоль и с помощью команды export [добавьте переменные окружения](#).
- Перейдите в каталог **build**.
- Выполните последовательно следующие команды:

- Windows

```
1 cmake ../my_project/tests/mock_test_suite/src ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-
  i686.cmake"
```

- Linux и macOS на Intel (x86_64):

```
1 cmake ../my_project/tests/mock_test_suite/src \
2 -DSBIS_UNIT_TESTS=1 -G Ninja
```

- macOS на Apple M1 (ARM):

```
1 cmake ../my_project/tests/mock_test_suite/src \
2 -DSBIS_ARCHITECTURE=arm64 \
3 -DSBIS_UNIT_TESTS=1 -G Ninja
```

- Запуск сборки проекта.

```
ninja
```

- Запуск тестирования проекта. Команды написаны для SBIS SDK 23.5100.

- Windows

```
1 py -3.11-32 "%SBISPlatformSDK_235100%\tools\test_framework"
  "..\my_project\tests" ^
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"
```

- Linux

```
1 /opt/python3.11/bin/python3.11
  $SBISPlatformSDK_235100/tools/test_framework
  ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

- macOS

```
1 python3.11 $SBISPlatformSDK_235100/tools/test_framework
  ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```